Classroom Lecture

Getting RFID Working on the Raspberry Pi Pico

with VS Code & MicroPython

From wiring the sensor to reading and writing RFID cards

Subject	Level	Duration	Board
RPi Programming and Application	Beginner – Intermediate	90 – 120 min	Pico / Pico W

ℹ Based on / Improved from

- Original source: Random Nerd Tutorials — 'Programming Raspberry Pi Pico with VS Code and MicroPython'
- This lecture extends that foundation to cover SPI-based RFID hardware (MFRC522 module), library installation, UID reading, block read/write, and a practical access-control project.
- Expanded with instructor notes, concept tables, troubleshooting, checkpoints, and extension tasks.

In this guide, you will learn how to connect an MFRC522 RFID reader module to the Raspberry Pi Pico, install a MicroPython driver library, read card UIDs, read and write data blocks, and build a simple access-control project — all using VS Code with the MicroPico extension.

1. Lesson Overview & Learning Objectives

By the end of this lesson every student will be able to:

- Explain what RFID is and how the MFRC522 module communicates with the Pico over SPI.
- Wire a MFRC522 module to the Pico correctly using the SPI0 bus.
- Install and import a MicroPython MFRC522 library from a .py source file.
- Read the unique identifier (UID) from an RFID card or key fob.
- Read and write data to a Mifare Classic 1K card block.
- Build a simple access-control system that grants or denies entry based on a stored UID.
- Diagnose and fix the eight most common RFID setup problems.

Instructor Opening (5 min)

- Ask: 'Where have you encountered RFID or NFC in real life?' — students will mention door fobs, contactless payment cards, library books, pet microchips, passports.
- Show a card and key fob. Ask what they have in common (antenna coil + chip).
- Demo: swipe a pre-programmed card and print its UID to the REPL. Ask students: 'What can we do with this number?'

2. Background Concepts — What Is RFID?

2.1 RFID in Plain English

RFID stands for Radio-Frequency Identification. An RFID system always has two components:

- A reader (the MFRC522 module) — generates a 13.56 MHz electromagnetic field using its antenna coil.
- A tag (card or key fob) — contains a tiny chip and a printed antenna. When brought close to the reader, the field induces enough current in the tag's antenna to power the chip, which then transmits data back wirelessly — no battery needed.

The MFRC522 is one of the most popular RFID reader ICs in the world. It supports the ISO 14443A / Mifare standard — the same standard used in most building access cards, transport cards, and many NFC-enabled devices.

Key Concept — RFID vs NFC

- RFID (Radio-Frequency Identification) is the broad technology — any wireless ID system.
- NFC (Near-Field Communication) is a subset of RFID operating at 13.56 MHz over very short distances (<4 cm). The MFRC522 handles NFC cards.
- Mifare Classic 1K is a specific NFC card type with 1 KB of storage split into 16 sectors of 4 blocks each. Each block holds 16 bytes.

2.2 How the MFRC522 Talks to the Pico — SPI

The MFRC522 uses SPI (Serial Peripheral Interface) to communicate with the Pico — a fast, four-wire synchronous protocol. Students should know the four signal lines:

SPI Signal	MFRC522 Pin	Description
SCK	SCK	Clock — generated by the Pico (master)
MOSI	MOSI	Master Out Slave In — data from Pico to sensor
MISO	MISO	Master In Slave Out — data from sensor to Pico
CS / SDA	SDA	Chip Select — pulled LOW to select the device
RST	RST	Hardware reset — active LOW
3.3V / GND	3.3V / GND	Power supply (do NOT connect to 5V — damages the IC)

⚠ Critical: 3.3V Only!

- The MFRC522 operates at 3.3V logic. Connecting it to 5V will destroy the IC.
- The Raspberry Pi Pico GPIO pins are 3.3V — safe to connect directly.
- The 3V3(OUT) pin (pin 36) on the Pico provides regulated 3.3V — use this for VCC.

2.3 Mifare Classic 1K Memory Layout

Each Mifare Classic 1K card is organised as 16 sectors. Each sector has 4 blocks. Block 0 of Sector 0 contains the manufacturer data (read-only UID). The last block of every sector is the Sector Trailer — it stores the two 6-byte authentication keys (Key A and Key B) and access conditions. All other blocks are freely readable and writable after authentication.

Block	Contents	Notes
Sector 0, Block 0	Manufacturer data + UID	Read-only — factory programmed
Sector 0, Blocks 1-2	User data (16 bytes each)	Read/Write after auth with Key A
Sector 0, Block 3	Sector Trailer (Keys + access)	Default Key A = FF FF FF FF FF FF
Sectors 1-15, Blocks 0-2	User data	16 bytes per block
Sectors 1-15, Block 3	Sector Trailer	Default keys same as above

3. Step 1 — Hardware & Wiring

3.1 Components Required

Component	Quantity	Notes
Raspberry Pi Pico or Pico W	1	MicroPython firmware already flashed
MFRC522 RFID Reader Module	1	Includes antenna PCB and header pins
Mifare Classic 1K card	1+	White card (often included with module)
Mifare key fob	1+	Optional — same protocol
Female-to-female jumper wires	7	Or male-to-female if breadboarding
USB cable (data-capable)	1	Power + serial communication to Pico
Breadboard	1	Optional but recommended for stability

3.2 Wiring the MFRC522 to the Pico

Connect the module using the table below. This lecture uses SPI0 on the Pico. All GPIO numbers refer to GP (pin function) numbers, not the physical header pin numbers.

MFRC522 Pin	Pico GP Pin	Physical Pin	Colour (suggested)
3.3V	3V3(OUT)	Pin 36	Red
GND	GND	Pin 38	Black
RST	GP22	Pin 29	Yellow
SDA (CS)	GP5	Pin 7	Orange
MOSI	GP7	Pin 10	Blue
MISO	GP4	Pin 6	Green
SCK	GP6	Pin 9	White

Instructor Tip

- Draw the wiring on the whiteboard BEFORE students wire up. Incorrect wiring is the #1 cause of module failure in class.
- Have students double-check: (a) VCC is on 3V3(OUT) not VSYS/5V, (b) SDA goes to GP5 not GP4.
- IRQ pin on the MFRC522 is NOT connected — interrupt-driven mode is out of scope for this lecture.

3.3 Verify Power with the LED

Before loading any code, plug the Pico into USB. The MFRC522 has a small green LED that should illuminate as soon as power is applied. If it does not light up, re-check the 3.3V and GND connections before proceeding.

✓ Checkpoint 1 — Hardware Wired

- MFRC522 LED is lit when Pico is connected to USB.
- All seven jumper wires are seated firmly.
- 3.3V (not 5V) is confirmed on the VCC pin.

4. Step 2 — Install the MicroPython MFRC522 Library

4.1 Why a Library Is Needed

MicroPython's standard library does not include a driver for the MFRC522. We must install a third-party driver that wraps the low-level SPI commands into convenient functions. The driver we use is a lightweight single-file library: mfrc522.py.

ℹ About the Library

- Source: github.com/micropython-lib/micropython-lib (community-maintained).
- It is a single Python file (mfrc522.py) — no package manager required.
- Provides: init_spi(), request(), anticoll(), auth(), read(), write(), stop_crypto1().
- Compatible with both Pico and Pico W without modification.

4.2 Download and Upload the Library

1. On your PC, download the mfrc522.py file from:

```
https://raw.githubusercontent.com . . . .
drivers/mfrc522/mfrc522.py
```

2. Save the file as mfrc522.py in your project folder alongside your other .py files.
3. In VS Code, open the Command Palette (Ctrl+Shift+P) and run: MicroPico: Upload current file to Pico.
4. Alternatively, right-click mfrc522.py in the Explorer panel and select Upload file to Pico.
5. Verify the upload: in the REPL terminal type:

```
import os
print(os.listdir())  # mfrc522.py should appear in the list
```

⚠ Alternative: Upload Entire Project

- If your project folder has both main.py and mfrc522.py, use MicroPico: Upload project to Pico to copy all files at once.
- This is faster for multi-file projects but ensure you are not accidentally overwriting an existing main.py you want to keep.

✓ Checkpoint 2 — Library Uploaded

- mfrc522.py appears in os.listdir() output in the REPL.
- No ImportError when running: from mfrc522 import MFRC522

5. Step 3 — Reading a Card UID

5.1 How UID Reading Works

When a card enters the MFRC522 field, the reader sends a Request command. The card replies with its ATQA (Answer To reQuest, type A) code. The reader then sends an Anticollision command to retrieve the card's UID — a 4 or 7 byte unique identifier printed (as hex) on most cards. No authentication is needed to read the UID — it is transmitted in plain text.

5.2 The Code — read_uid.py

Create a new file called read_uid.py in your project folder and type the following:

```
from mfrc522 import MFRC522
import utime

reader = MFRC522(
    sck=6,
    mosi=7,
    miso=4,
    rst=22,
    cs=5,
    spi_id=0
)

print("Bring TAG closer...")
print("")

while True:
    reader.init()
    (stat, tag_type) = reader.request(reader.REQIDL)
    if stat == reader.OK:
        (stat, uid) = reader.SelectTagSN()
        if stat == reader.OK:
            card = int.from_bytes(bytes(uid),"little",False)
            print("CARD ID: "+str(card))

    utime.sleep_ms(500)    # ☑ INSIDE loop
```

5.3 Code Walkthrough

Walk through each important line with the class before running:

Code	Explanation
`MFRC522(sck=…)`	Creates the driver. Pins match the wiring from Step 1. SPI0 is selected automatically from the GP numbers.
`reader.request(REQIDL)`	Broadcasts a 'wake up' command. Returns OK if any card is detected in the field.
`reader.anticoll()`	Retrieves the UID bytes. Handles multiple cards in the field simultaneously.
`':'.join([…format…])`	Formats the raw bytes as a colon-separated hex string e.g. A3:04:BB:12.
`sleep(1) after detect`	Prevents the same card printing dozens of times — one print per second while held near.
`try / except KeyboardInterrupt`	Lets Ctrl+C cleanly stop the loop without a traceback error filling the REPL.

5.4 Run the File

1. Ensure the status bar shows MicroPico Connected.
2. Click the Run button (▶) in the VS Code status bar.
3. Hold an RFID card or key fob close to the module antenna.
4. The REPL should print: Card detected! UID = A3:04:BB:12 (your numbers will differ).
5. Try a second card — confirm the UIDs are different.
6. Press Stop (■) when done.

✓ Checkpoint 3 — UID Reading

- A UID is printed to the REPL each time a card is held near the module.
- Different cards print different UIDs.
- Program stops cleanly with no error message when Stop is pressed.

6. Step 4 — Reading and Writing Card Data

6.1 Authentication Requirement

To read or write user data blocks, you must first authenticate against the relevant sector using a key. Fresh cards from the manufacturer have all sectors set to the default key: FF FF FF FF FF FF (6 bytes of 0xFF). We use Key A throughout this exercise.

> ⚠ **Authentication Error is Common**
>
> - If authentication fails, read/write will raise an exception. Most common causes:
>   (a) Wrong key — the card has been previously written with a custom key.
>   (b) Card moved away during operation — keep the card very still.
>   (c) Block 3 (Sector Trailer) targeted — this block stores keys; use blocks 1 or 2 instead.

6.2 Reading a Data Block — read_block.py

This example reads Block 1 of Sector 0:

```
# read_block.py
# Authenticate and read 16 bytes from Block 1, Sector 0

from mfrc522 import MFRC522
import utime

# Default key (Factory key for most RFID cards)
DEFAULT_KEY = [0xFF]*6

# Initialize RFID
reader = MFRC522(
    sck=6,
    mosi=7,
    miso=4,
    rst=22,
    cs=5,
    spi_id=0
)

print("=== RFID READ BLOCK ===")
print("Place card near reader...\n")

while True:
    reader.init()

    # Step 1: Request card
    (stat, tag_type) = reader.request(reader.REQIDL)
```

```
    if stat == reader.OK:
        print("Card detected!")

        # Step 2: Get UID
        (stat, uid) = reader.SelectTagSN()

        if stat == reader.OK:
            print("UID:", uid)

            # Step 3: Authenticate block
            block_addr = 1  # Block 1 (Sector 0)

            if reader.auth(reader.AUTHENT1A, block_addr, DEFAULT_KEY, uid) == reader.OK:
                print("Authentication success")

                # Step 4: Read block
                (stat, data) = reader.read(block_addr)

                if stat == reader.OK:
                    print("Raw Data:", data)

                    # Convert to string
                    text = ''.join([chr(i) if 32 <= i <= 126 else '.' for i in data])
                    print("Text:", text)

                else:
                    print("Read failed")

                reader.stop_crypto1()

            else:
                print("Authentication error")

    utime.sleep_ms(500)
```

stop_crypto1() Is Critical

- After any authenticated read or write session, you MUST call reader.stop_crypto1().
- If you forget, the module stays in encrypted mode and the next request() will fail silently.
- Make it a habit: auth → read/write → stop_crypto1(). Always in that order.

6.3 Writing a Data Block — write_block.py

This example writes a 16-byte message to Block 1 of Sector 0:

```
# write_block.py
# Write a 16-byte string to Block 1, Sector 0

from mfrc522 import MFRC522
import utime

DEFAULT_KEY = [0xFF]*6

reader = MFRC522(
    sck=6,
    mosi=7,
    miso=4,
    rst=22,
    cs=5,
    spi_id=0
)

print("=== RFID WRITE BLOCK ===")
print("Place card near reader...\n")

# Data to write (must be EXACTLY 16 bytes)
text = "00 SomnangMeanLeap"
data = [ord(c) for c in text]

# Pad to 16 bytes
while len(data) < 16:
    data.append(32)  # space

while True:
    reader.init()

    (stat, tag_type) = reader.request(reader.REQIDL)

    if stat == reader.OK:
        print("Card detected!")

        (stat, uid) = reader.SelectTagSN()

        if stat == reader.OK:
            print("UID:", uid)
```

```
            block_addr = 1

            if reader.auth(reader.AUTHENT1A, block_addr, DEFAULT_KEY, uid) == reader.OK:
                print("Authentication success")

                # Write data
                if reader.write(block_addr, data) == reader.OK:
                    print("Write successful!")

                    # Verify by reading
                    (stat, new_data) = reader.read(block_addr)
                    if stat == reader.OK:
                        print("Verify:", new_data)

                else:
                    print("Write failed")

                reader.stop_crypto1()

            else:
                print("Authentication error")

    utime.sleep_ms(1000)
```

After running write_block.py, run read_block.py again on the same card — you should see your message printed as text in the REPL.

✓ Checkpoint 4 — Read / Write Working

- read_block.py prints raw bytes and decoded text from a card block.
- write_block.py writes a 16-character string without errors.
- Running read_block.py after writing confirms the new data is stored on the card.

7. Step 5 — Practical Project: Access Control

7.1 Project Overview

Students will build a simple access-control system. The Pico stores a list of authorised card UIDs in a Python list. When a card is scanned, its UID is checked against the list. If the UID matches, the onboard LED flashes green (fast) and 'ACCESS GRANTED' is printed. If it does not match, the LED flashes slowly and 'ACCESS DENIED' is printed.

🎯 **System Behavior**

- Scan RFID card
- Read **UID + (optional) stored text**
- Compare with authorized list
- Output result:
-

Condition	Action
✅ Authorized	Fast LED blink + “ACCESS GRANTED”
❌ Not Authorized	Slow LED blink + “ACCESS DENIED”

Components:

- Raspberry Pi Pico / Pico W
- RC522 RFID Module
- LED (optional external, or use onboard LED GP25)

> 📋 **Instructor Setup**
>
> - Before the exercise, have each student scan their card with ex6.2_read_block.py and note their UID.
> - In the project code below, students replace the placeholder UID with their own.
> - • Optional extension: store authorised UIDs in a file on the Pico (see Extension E2).

🧠 System Logic

Scan Card → Read UID → Compare → Decision → LED + Message

```
IF UID in authorized_list:
    ACCESS GRANTED
    LED blink FAST
ELSE:
    ACCESS DENIED
    LED blink SLOW
```

7.2 The Code — access_control.py

```
from mfrc522 import MFRC522
from machine import Pin
import utime

# ========================
# LED SETUP (Onboard LED)
# ========================
led = Pin(25, Pin.OUT)

# ========================
# AUTHORIZED DATABASE
# ========================
# Example: store UID + name
AUTHORIZED_CARDS = [
    {"uid": [1, 192, 242, 6], "name": "Student A"},
    {"uid": [5, 186, 101, 6], "name": "Student B"},
]

# ========================
# RFID SETUP
# ========================
reader = MFRC522(
    sck=6,
    mosi=7,
    miso=4,
    rst=22,
    cs=5,
    spi_id=0
)

print("=== ACCESS CONTROL SYSTEM ===")
print("Scan your card...\n")
# ========================
# LED FUNCTIONS
# ========================
def access_granted():
    print("ACCESS GRANTED ☑")
    for _ in range(5):
        led.value(1)
        utime.sleep_ms(100)
        led.value(0)
        utime.sleep_ms(100)
```

```
def access_denied():
    print("ACCESS DENIED ❌")
    for _ in range(3):
        led.value(1)
        utime.sleep_ms(500)
        led.value(0)
        utime.sleep_ms(500)


# ========================
# CHECK AUTHORIZATION
# ========================
def is_authorized(uid):
    for card in AUTHORIZED_CARDS:
        if card["uid"] == uid:
            return True, card["name"]
    return False, None


# ========================
# MAIN LOOP
# ========================
while True:
    reader.init()

    (stat, tag_type) = reader.request(reader.REQIDL)

    if stat == reader.OK:
        print("\nCard detected!")

        (stat, uid) = reader.SelectTagSN()

        if stat == reader.OK:
            print("UID:", uid)

            authorized, name = is_authorized(uid)

            if authorized:
                print("Welcome,", name)
                access_granted()
            else:
                access_denied()

    utime.sleep_ms(500)
```

7.3 Deploying as main.py

7. Rename access_control.py to main.py (or save a copy as main.py).
8. Right-click main.py in VS Code Explorer and select Upload file to Pico.
9. Unplug the Pico and reconnect it to a USB power bank.
10. Test: approved card → fast triple blink. Unknown card → single slow blink.

> ✓ **Checkpoint 5 — Access Control Working**
> - Authorised card triggers 'ACCESS GRANTED' and fast triple LED blink.
> - Unknown card triggers 'ACCESS DENIED' and single slow blink.
> - System runs standalone from a power bank — no computer required.

🧪 7.4 Expected Output

✅ Authorized Card

```
Card detected!
UID: [12, 34, 56, 78]
Welcome, Student A
ACCESS GRANTED
```

❌ Unauthorized Card

```
Card detected!
UID: [99, 88, 77, 66]
ACCESS DENIED
```

🔐 Enhanced Security Logic (UID + Data Match)

To push this further—**UID-only checks are weak** (UIDs can be cloned). A better design is to require **two factors**:

1. **UID match** (card identity)
2. **Stored data match** (e.g., name/ID saved on the card)

Below is a clean upgrade your students can implement.

```
Scan card → Read UID → Authenticate → Read Block → Extract Name
      ↓
Check UID + Name together → Decision
```

🧠 Idea

- Store a **name (or student ID)** in **Block 1**
- Keep a database in Pico:
  UID + NAME must BOTH match

💾 Example Data on Card (Block 1)

"Somnang_2026 " (16 bytes)

💻 Enhanced Code — access_control_secure.py

```
from mfrc522 import MFRC522
from machine import Pin
import utime

# ========================
# LED SETUP
# ========================
led = Pin(25, Pin.OUT)

# ========================
# DEFAULT KEY
# ========================
DEFAULT_KEY = [0xFF]*6

# ========================
# AUTHORIZED DATABASE
# UID + NAME (must match both)
# ========================
AUTHORIZED_CARDS = [
    {"uid": [12, 34, 56, 78], "name": "Somnang_2026"},
    {"uid": [111, 222, 333, 444], "name": "STUDENT_B"},
    {"uid": [1, 192, 242, 6], "name": "Student A"},
    {"uid": [5, 186, 101, 6], "name": "Student D"},
]

# ========================
# RFID SETUP
# ========================
reader = MFRC522(
    sck=6,
    mosi=7,
    miso=4,
    rst=22,
    cs=5,
    spi_id=0
)

print("=== SECURE ACCESS CONTROL ===")
print("Scan your card...\n")
```

```
# ========================
# LED FUNCTIONS
# ========================
def access_granted():
    print("ACCESS GRANTED ☑")
    for _ in range(5):
        led.value(1)
        utime.sleep_ms(100)
        led.value(0)
        utime.sleep_ms(100)

def access_denied():
    print("ACCESS DENIED ❌")
    for _ in range(3):
        led.value(1)
        utime.sleep_ms(500)
        led.value(0)
        utime.sleep_ms(500)

# ========================
# READ TEXT FROM BLOCK
# ========================
def read_name_from_card(uid, block=1):
    if reader.auth(reader.AUTHENT1A, block, DEFAULT_KEY, uid) == reader.OK:
        (stat, data) = reader.read(block)
        reader.stop_crypto1()

        if stat == reader.OK:
            # Convert to string
            text = ''.join([chr(i) for i in data]).strip()
            return text
    return None

# ========================
# CHECK AUTHORIZATION
# ========================
def is_authorized(uid, name):
    for card in AUTHORIZED_CARDS:
        if card["uid"] == uid and card["name"] == name:
            return True
    return False
```

```
# ========================
# MAIN LOOP
# ========================
while True:
    reader.init()

    (stat, tag_type) = reader.request(reader.REQIDL)

    if stat == reader.OK:
        print("\nCard detected!")

        (stat, uid) = reader.SelectTagSN()

        if stat == reader.OK:
            print("UID:", uid)

            # Read stored name
            name = read_name_from_card(uid)

            if name:
                print("Card Name:", name)

                if is_authorized(uid, name):
                    access_granted()
                else:
                    access_denied()
            else:
                print("Failed to read card data")
                access_denied()

    utime.sleep_ms(500)
```

🔒 Why This Is More Secure

Method	Security
UID only	❌ Easy to clone
UID + Name	✅ Stronger
UID + Name + Key	🔥 Even better

🚀 Suggested Advanced Upgrades (For Your Course)

1️⃣ Add Secret Key (Triple Check)

```
{"uid": [...], "name": "...", "key": "A1B2"}
```

2️⃣ Encrypt Data (Simple XOR for students)

3️⃣ Store:

- Student ID
- Expiry date
- Access level

🎓 Student Exercise Ideas

Task 1:

Write name to RFID card

Task 2:

Read and verify UID + name

Task 3:

Modify system:

- Deny if name mismatch
- Log attempts

🧠 Teaching Insight

This upgrade introduces:

- Authentication concepts
- Data integrity
- Multi-factor verification

👉 This is a **real-world embedded security system**

8. Troubleshooting — Common Problems & Fixes

Review this table with students before lab time. The majority of issues fall into one of these eight categories:

Symptom	Likely Cause	Fix
MFRC522 LED does not light	VCC wired to 5V (VSYS) or wrong pin	Move VCC wire to 3V3(OUT) — pin 36 on the Pico.
ImportError: no module named 'mfrc522'	mfrc522.py not uploaded to Pico	Upload mfrc522.py via MicroPico. Verify with os.listdir().
reader.request always returns not-OK	Wiring error on SPI pins	Double-check SCK=GP6, MOSI=GP7, MISO=GP4, CS=GP5.
Card not detected / intermittent	Card held too far or at an angle	Hold card flat and within 3 cm of the antenna coil.
Authentication failed every time	Card has custom keys or is not Mifare Classic	Try default key FF×6. Check card type — Mifare Ultralight has no auth.
stop_crypto1 not called error	Auth session left open from previous run	Press Ctrl+C, then Ctrl+D (soft reset) in REPL. Restart the script.
Written data reads back as zeros	Data list is wrong length (must be exactly 16 bytes)	Pad string to exactly 16 characters: MESSAGE = 'Hello! '
Multiple cards confuse the reader	Only one card should be in field at a time	Remove all but one card. anticoll() handles one card per request.

8.1 REPL Diagnostic Commands

When something goes wrong, use these commands to inspect the state of the Pico and the module:

```
# Check what files are on the Pico
import os
print(os.listdir())

# Soft-reset the Pico (clears any stuck crypto session)
# Press Ctrl+D in the REPL

# Quick SPI connection test (should not raise an error)
from machine import SPI, Pin
spi = SPI(0, baudrate=1000000, polarity=0, phase=0,
          sck=Pin(6), mosi=Pin(7), miso=Pin(4))
print('SPI init OK:', spi)

# Check free RAM
import gc; gc.collect()
print(gc.mem_free(), 'bytes free')
```

9. Extension Challenges

For students who complete the main exercises ahead of schedule. Intentionally minimal scaffolding.

#	Challenge	Description	Hint
E1	Card Logger	Scan a card and append its UID and a timestamp (using machine.RTC) to a log.txt file stored on the Pico. Print all log entries to the REPL on startup.	Use open('log.txt','a') and machine.RTC().datetime() for timestamps.
E2	File-Based Allow-list	Instead of hard-coding UIDs in the Python list, read them from a plain-text file (authorised.txt) on the Pico — one hex UID per line. Reload the file on each scan.	Use open('authorised.txt','r').readlines() and .strip() to parse.
E3	Buzzer Feedback	Add an active buzzer (or passive buzzer with PWM) on GP16 that beeps once for granted and twice for denied.	from machine import PWM. Use duty_u16() and freq() to control tone.
E4	Write & Read Name	Allow the user to type their name in the REPL (input()). Write it to the card. Then read it back from a different Pico and greet the card holder by name.	Combine write_block.py and read_block.py. Pad/truncate to 16 bytes.
E5	Sector Scan	Read all 16 blocks of Sector 0 to 3 and display them in a formatted table in the REPL.	Loop BLOCK from 0 to 15. Re-authenticate at each sector boundary (blocks 0, 4, 8, 12).

10. Teacher Notes & Discussion Guide

10.1 Lesson Timing

Time	Activity	Format
0 – 10 min	Section 1 & 2: Intro, RFID concepts, show module and cards	Whole class
10 – 20 min	Section 3: Wiring. Demonstrate on doc-cam first, then students wire up	Pairs
20 – 30 min	Section 4: Upload mfrc522.py library. Verify with os.listdir()	Individual
30 – 50 min	Section 5: Write and run read_uid.py. Students note their own card UID	Individual
50 – 70 min	Section 6: Read/write block exercises. Walk through code first	Individual
70 – 90 min	Section 7: Access control project. Students enter own UIDs	Individual / Pairs
90 – 105 min	Section 9: Extension challenges for early finishers	Individual
105 – 120 min	Debrief, show-and-tell, discussion questions (Section 10.2)	Whole class

10.2 Discussion Questions

11. The card's UID is transmitted without encryption. What security implications does this have? How is this addressed in real-world access control systems?
12. Mifare Classic 1K uses a proprietary algorithm (Crypto-1) for sector authentication that has known cryptographic weaknesses. Does this matter for a school door-entry system? When does it matter?
13. We stored UIDs in a Python list inside the code. What are the disadvantages? (Students should arrive at: changing the list requires re-uploading code.) How would a file-based or server-based approach improve this?
14. The internal MFRC522 antenna range is about 3–5 cm. In what scenarios is this short range a feature rather than a limitation?
15. If you needed the Pico to log card scans to a cloud database, what additional hardware and code would be required? (Leads to: Pico W, Wi-Fi, HTTP/MQTT, server-side database.)

10.3 Differentiation

- Support: Provide a printed wiring diagram and pre-typed code files. Pair struggling students with a peer mentor. Focus on Sections 3–5 only.
- Core: Complete all exercises in Sections 3 through 7 as written.
- Extension: Challenges E1–E3 (logging, file-based allow-list, buzzer feedback).
- Advanced: E4 and E5 (cross-Pico name read, full sector scan), plus challenge students to research Mifare DESFire cards and explain why they are more secure than Classic 1K.

11. Quick-Reference Card

Print or display at each workstation.

MFRC522 Wiring Summary	Key MFRC522 Library Calls

3.3V → 3V3(OUT) pin 36
GND → GND pin 38
RST → GP22 pin 29
SDA → GP5 pin 7
MOSI → GP7 pin 10
MISO → GP4 pin 6
SCK → GP6 pin 9
	`MFRC522(sck,mosi,miso,cs,rst)`
	→ Initialise reader with SPI pins
	`reader.request(REQIDL)`
	→ Detect card in field
	`reader.anticoll()`
	→ Retrieve card UID bytes
	`reader.auth(AUTHENT1A, blk, key, uid)`
	→ Authenticate before read/write
	`reader.read(block)`
	→ Read 16 bytes from block
	`reader.write(block, data_16_bytes)`
	→ Write exactly 16 bytes to block
	`reader.stop_crypto1()`
	→ **End auth session — ALWAYS call!**

— End of Lecture Notes —